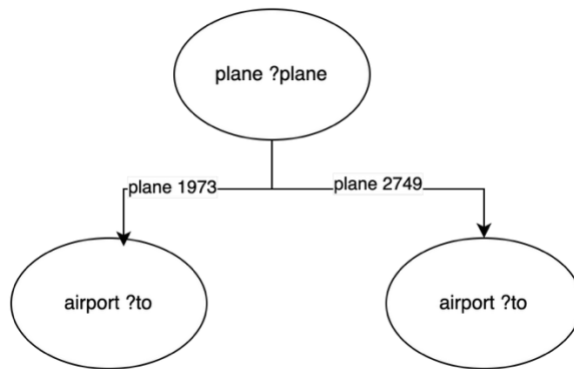
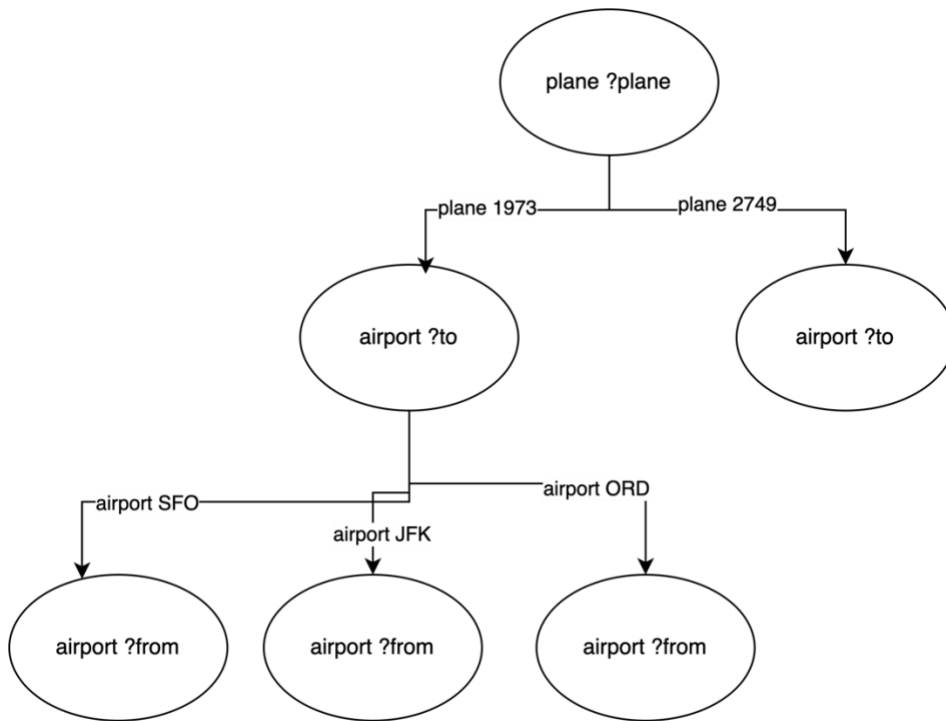


- 1) To check if the fly action is permissible, we could iterate through each precondition and then try to unify it with each predicate in the state. If we can't unify the precondition with any predicate, the state is not permissible. Starting with the first precondition, we attempt to unify (plane ?plane) and (plane 1973). Since they unify, with the substitution list {?plane: 1973}, we continue with the next precondition. Next, we try to unify (airport ?to) and (airport SFO), which unifies with {?to : SFO}. Next, we try to unify (airport ?from) and (airport JFK), which unifies with {?from : JFK}. Next, we try to unify (at ?plane ?from) and (at 1973 SFO), which unifies with {?plane: 1973 , ?from: SFO}. Since, all preconditions were unified the action is permissible.
- 2) To check all possible unification strategies, we can use DFS to branch off at each precondition that can be unified in multiple ways. We start with the first precondition (plane ?plane) and iterate through all the predicates to check for unification. Doing this we see that the precondition can unify with (plane 1973) and (plane 2749). This creates two branches to search through:

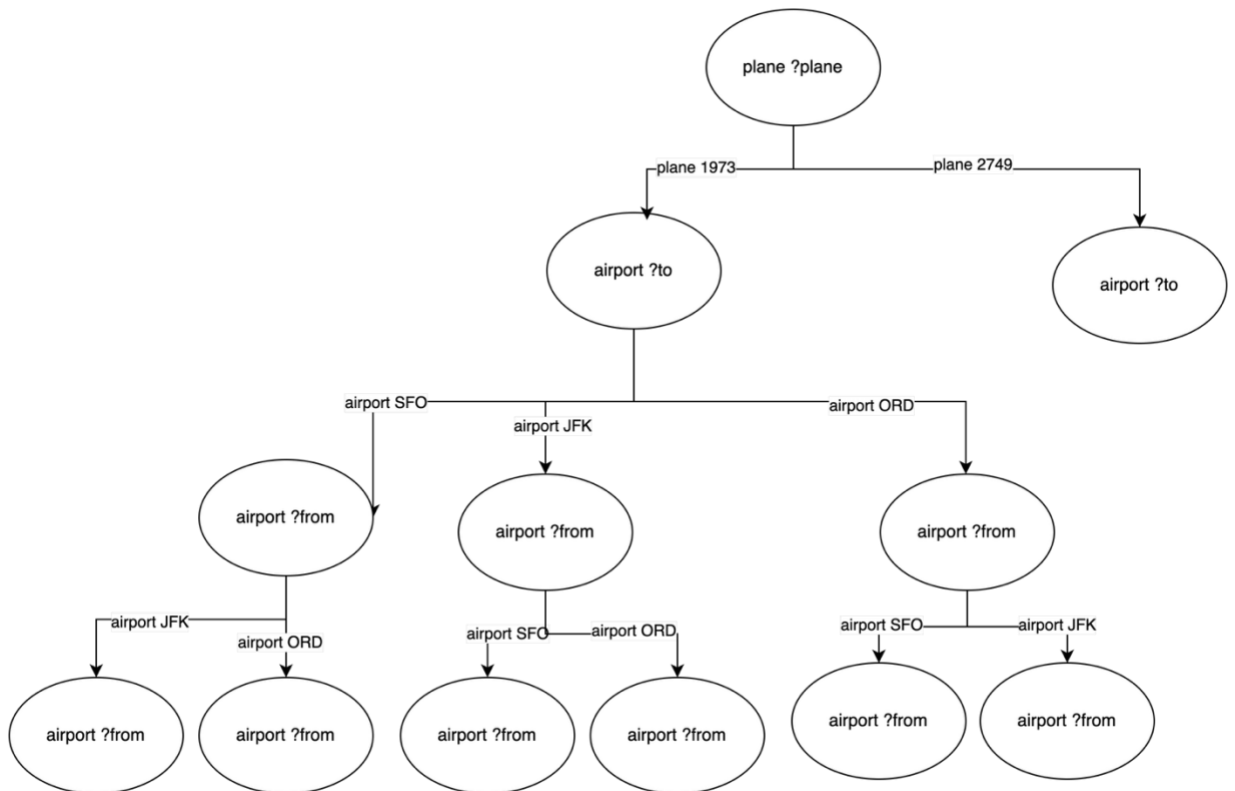


Next we continue with left branch and try to unify (airport ?to). We can unify this condition with (airport SFO), (airport JFK), (airport ORD), resulting in the following

graph:



Next, we can continue down the graph and unify (airport ?from) with the remaining airports for each branch, resulting in (note the leaf nodes should read (at ?plane ?from)):



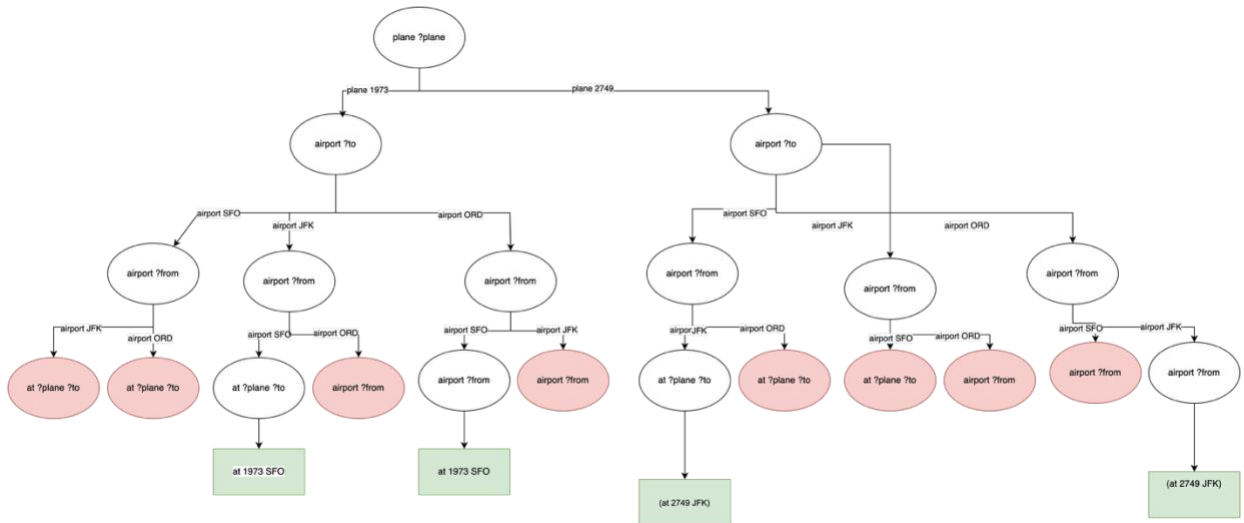
Next, we reach the final condition, (at ?plane ?from), by looping through all the current leaf nodes and remaining predicates we see that only (at 1973 SFO) unifies, which leaves us with (fly 1973 SFO JFK) and (fly 1973 SFO ORD).



Continuing from the 2749 branch, we can unify (airport ?from) with each airport.

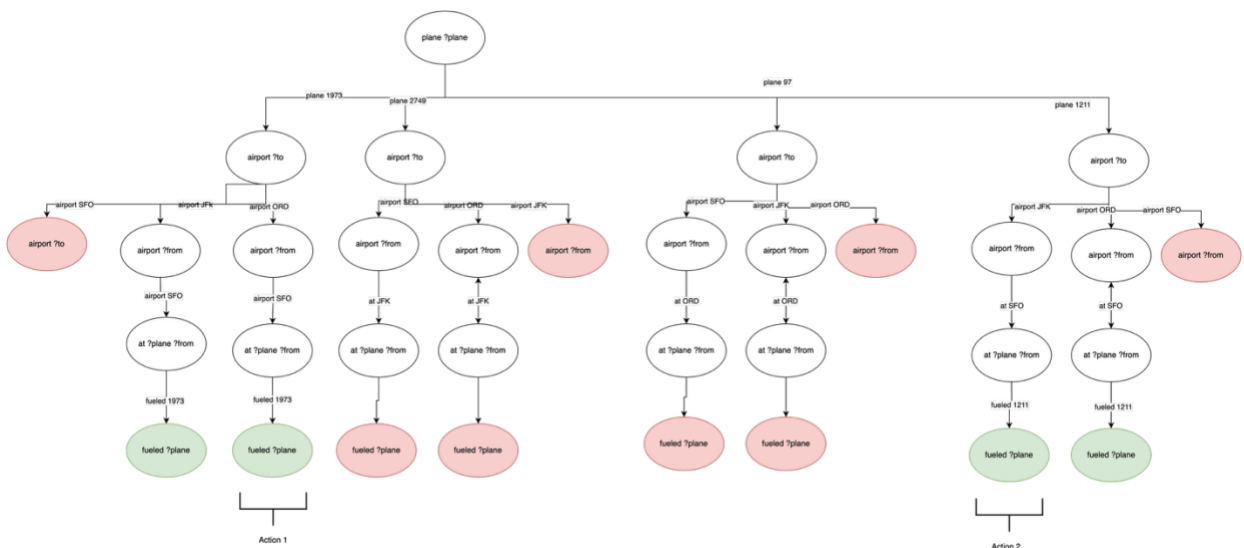


Next, we can unify (plane ?to) with the remaining airports in each branch. We follow the same process as before and unify (at ?plane ?from) which substitutes to (at 2749 JFK). This leaves only (fly 2749 JFK SFO) and (fly 2749 JFK ORD).



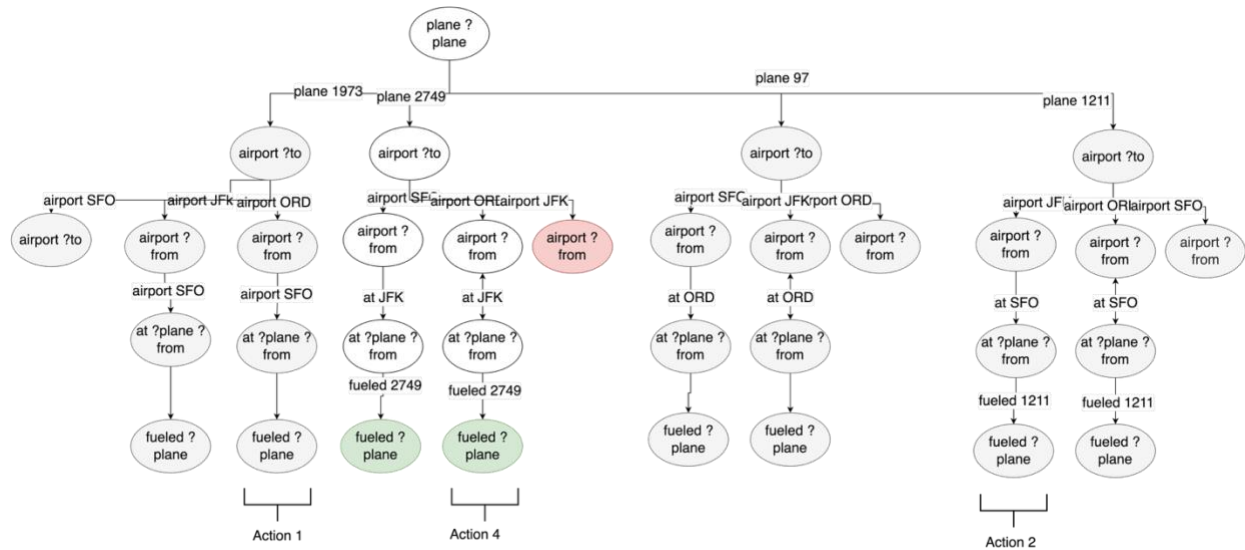
This shows that graph-based search is needed to fully enumerate all possible unification strategies.

- 3) Following the steps from part 2, we can define the search tree for each action, and then use that to select the action to take.



The first 2 actions that fulfil goal states in the graph are (fly 1973 SFO ORD) and (fly 1211 SFO JFK), so we add those to the plan, add their substituted add list to the state list, and remove their substituted delete list from the state. After taking the first two paths presented in the graph above,

we are left with no more unified actions for fly, so we can try to apply the action fuel. Since (plane ?plane) and (unfueled ?plane) unify with (plane 2749), (unfueled 2749), we can apply (fuel 2749) and modify the state to remove (unfueled 2749) and include, (fueled 2749). We can then continue searching for paths down the 2749 nodes, which leads us to the last action of (fly 2749 JFK SFO)



- 1) The capacity of each plane can be added to the state such as (holding 1973 100). We can then add goal states for airport such as (capacity SFO 100), which specifies that a 100-person plane must be at SFO airport. The capacity can then be added to/removed from the state as part of the add/del list of fly. For example, if 1973 is flying to JFK from SFO, the new state would include (capacity JFK 100) and (capacity SFO 100) would be removed.
- 2) A* could be used to guide the flight path of the planes scheduled by a forward planner.